

Algorithmique et Programmation orientée-objet

Ch.6 – Exceptions et Fichiers

Bruno Quoitin
(bruno.quoitin@umons.ac.be)

Table des Matières

- **Exception**

- Types
- Capture
- Déclenchement explicite
- Contrôlée ou pas ?
- Bonnes pratiques

- **Fichiers**

- Accès séquentiel (flux)
- Accès aléatoire

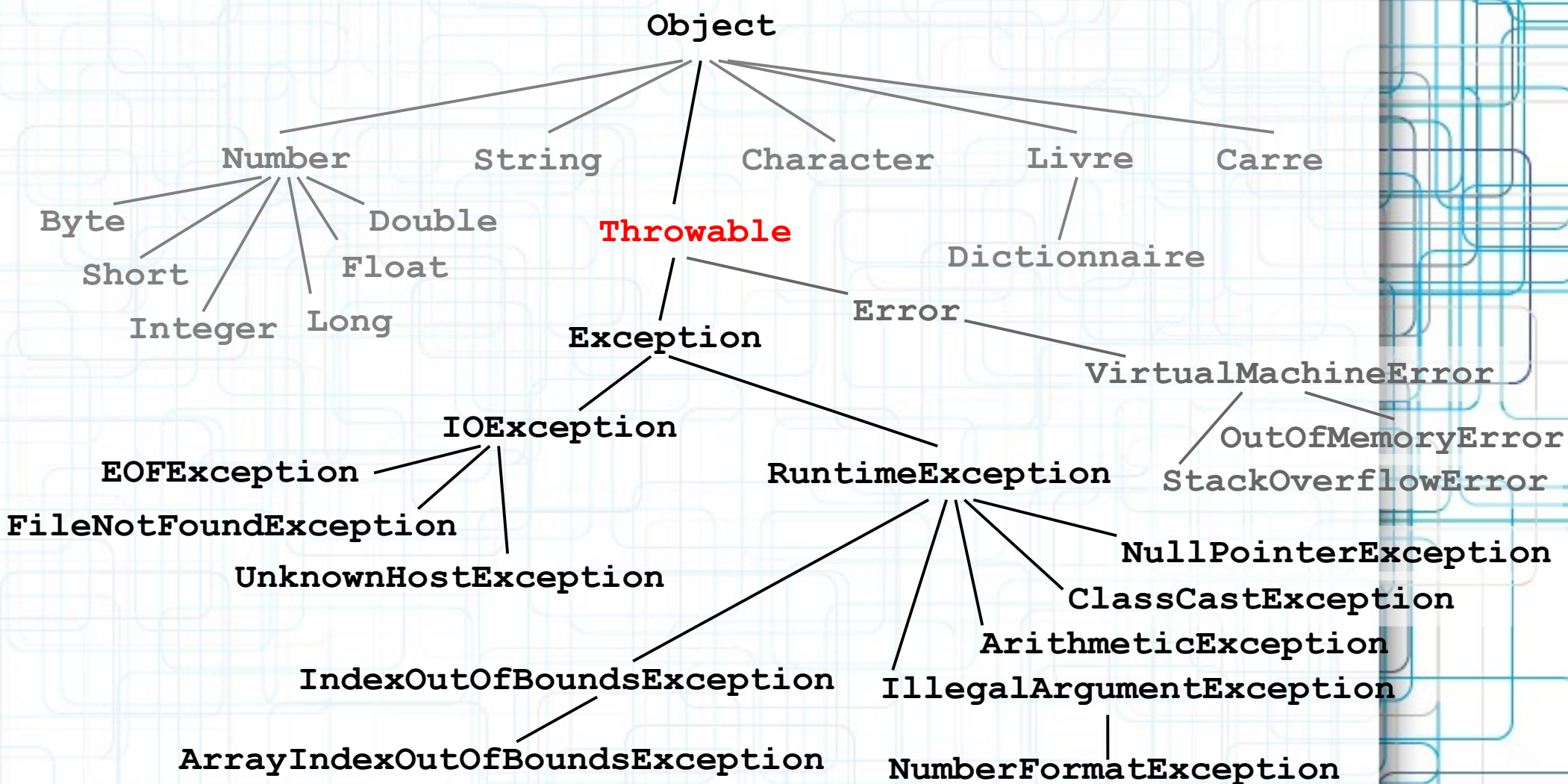
Exceptions

- **Introduction**

- Une **exception** est un événement qui se produit durant l'exécution d'un programme et qui en modifie le déroulement.
- Les exceptions sont utilisées pour gérer de manière contrôlée les *erreurs et autres situations inattendues* qui se produisent durant l'exécution.
- Les exceptions sont des **objets**. Différents types d'exceptions permettent de signaler des erreurs différentes et d'éventuellement leur associer des contextes différents tels qu'un message, voire un nom de fichier, une ligne dans un fichier, un argument ou une valeur erronés.

Exceptions

- Hiérarchie des classes d'exceptions



Exceptions

- **Classe Throwable**

Throwable

```
+getMessage()  
+printStackTrace()  
+getStackTrace()  
+toString()
```

- La classe `Throwable` est à la base de toutes les exceptions et erreurs. Elle fournit les services suivants :

- contient un message (d'erreur)

```
public String getMessage();
```

- permet de chaîner les exceptions

```
public String toString();
```

- une exception causée par une autre exception peut contenir une référence vers cette dernière.

- permet d'obtenir la trace de la pile d'exécution

```
public void printStackTrace();  
public StackTraceElement[] getStackTrace();
```

- liste des appels de méthodes qui ont conduit au déclenchement de l'exception.

Exceptions

- **Trace de la pile d'exécution**
 - La **trace de la pile d'exécution** indique les appels successifs de méthodes (et leur localisation dans le code source) qui ont mené au déclenchement de l'exception.
 - Exemple
 - L'**appel initial** est celui le plus bas (`main`) .
 - L'exception a eu lieu dans **l'appel le plus haut** (`forInputString`).

```
java.lang.NumberFormatException: For input string: "123toto"  
    at java.lang.NumberFormatException.forInputString(  
        NumberFormatException.java:48)  
    at java.lang.Integer.parseInt(Integer.java:458)  
    at java.lang.Integer.parseInt(Integer.java:499)  
    at MethExcPropag.maMethode(MethExcPropag.java:5)  
    at MethExcPropag.main(MethExcPropag.java:11)
```

Capture

- **Bloc try-catch-finally**

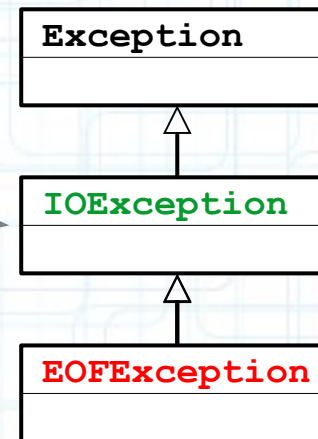
- Partie **try** entoure les instructions susceptibles de déclencher une exception
- Partie **catch** capture les exceptions d'un type
 - Peut apparaître plusieurs fois pour des types différents
 - Si types non-disjoints, ordonner du plus spécifique au moins spécifique !
- Partie **finally** toujours exécutée. Pratique si besoin de libérer des ressources (p.ex. fermeture d'un fichier)

```
var input = new Scanner(System.in);  
var x = input.nextInt();  
var y = input.nextInt();  
try {  
    System.out.println(x / y);  
} catch (ArithmeticException e) {  
    e.printStackTrace();  
}
```


Capture

- **Types d'exceptions non-disjoints**
 - Plusieurs clauses **catch** peuvent correspondre à une exception dans le cas où la classe mentionnée dans une clause **catch** est une sous-classe d'une classe mentionnée dans une autre clause **catch**.

```
try {  
    ...  
} catch (IOException e) {  
}  
} catch (EOFException e) {  
}
```



La seconde clause catch ne sera jamais utilisée car les instances de `EOFException` correspondent à la classe parent `IOException` dans la clause précédente !
Le compilateur génère une erreur dans ce cas.

Déclenchement explicite

- **Clause throws**

- Le mot-réservé **throw** est utilisé pour déclencher explicitement une exception. Il prend en argument une instance de l'exception déclenchée (p.ex. créée avec **new**).

```
if (x < 0)
    throw new IllegalArgumentException(
        "L'argument ne peut être négatif");
```

- Lorsqu'une exception est déclenchée, l'exécution du programme est déroutée. Les instructions qui suivent la clause **throw** ne sont pas exécutées. Soit l'exception est capturée par un gestionnaire d'exception, soit aucun gestionnaire d'exception n'est rencontré et le programme est terminé.

Table des Matières

- **Exception**

- Types
- Capture
- Déclenchement explicite
- Contrôlée ou pas ?
- Bonnes pratiques

- **Fichiers**

- Accès séquentiel (flux)
- Accès aléatoire

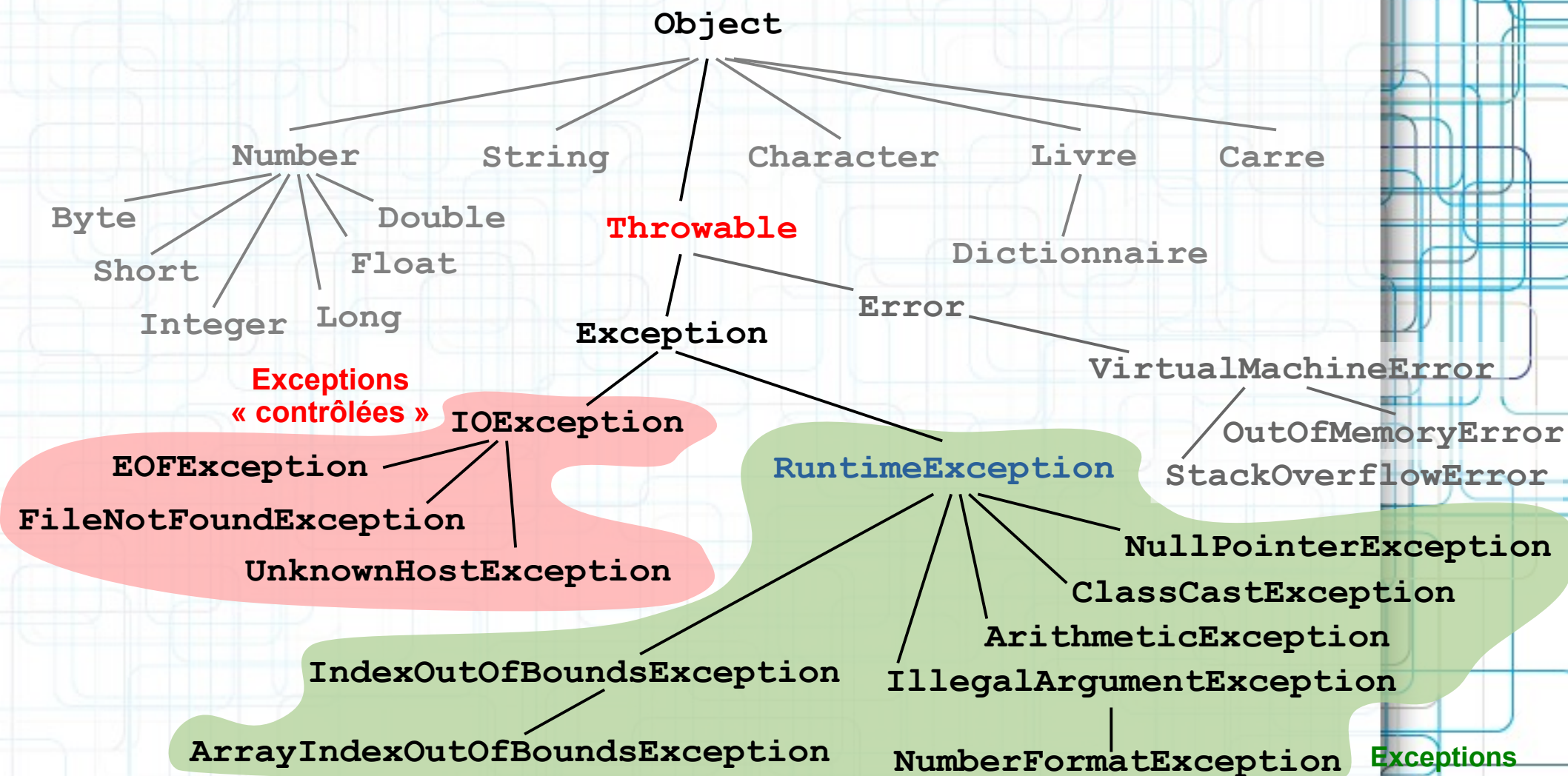
Contrôlée ou non ?

- **Checked vs Unchecked**

- Il existe deux catégories d'exceptions en Java
- **Exceptions « contrôlées »** (*checked exceptions*)
 - Prévues par un contrat établi entre le client et l'implémentation. Typiquement utilisées pour des erreurs récupérables.
 - Le client est forcé de vérifier la survenance de telles exceptions. Le compilateur vérifie que ces exceptions sont gérées.
- **Exceptions « non-contrôlées »** (*unchecked exceptions*)
 - Exceptions dont la survenance n'est pas explicitement prévue, comme par exemple la division par 0. Il s'agit en général d'erreurs de programmation, mais pas toujours.
 - Le compilateur n'effectue pas de vérification.
 - Sous-classes de `RuntimeException`.

Contrôlée ou non ?

- Hiérarchie des classes d'exceptions



Exceptions contrôlées

- **Déclaration des exceptions contrôlées**

- Une méthode susceptible de déclencher une exception contrôlée doit la **déclarer explicitement** en utilisant la directive **throws** suivie de la liste des exceptions concernées.

```
public class FileInputStream {  
  
    public FileInputStream(String name) throws FileNotFoundException  
    { /* ... */ }  
  
    public byte read() throws IOException  
    { /* ... */ }  
  
}
```

- Lorsqu'une méthode qui déclare qu'elle est susceptible de déclencher une exception est utilisée, le compilateur vérifie que
 - soit cette exception est capturée
 - soit la méthode utilisatrice devient à son tour susceptible de déclencher cette exception et doit le déclarer explicitement.

Exceptions contrôlées

- **Capture des exceptions contrôlées**

- La méthode ci-dessous ouvre un fichier pour en effectuer la lecture. Le constructeur de la classe `FileInputStream` est susceptible de déclencher une exception `FileNotFoundException`. La compilation de cette méthode déclenchera une **erreur à la compilation**.

```
public double[] lectureFichier(String filename) {  
    FileInputStream fis =  
        new FileInputStream(filename);  
    /* ... */  
}
```

- Solutions
 - capturer l'exception (**try-catch**)
 - ajouter une clause **throws**

Table des Matières

- **Exception**

- Types
- Capture
- Déclenchement explicite
- Contrôlée ou pas ?
- Bonnes pratiques

- **Fichiers**

- Accès séquentiel (flux)
- Accès aléatoire

Bonnes pratiques

- **Quand utiliser une exception ?**
 - L'utilisation d'exceptions doit rester... **exceptionnelle**... c'est-à-dire n'être utilisée que pour signaler une condition inhabituelle du programme, généralement dépendante de l'environnement ou des entrées/sorties.
 - entrée utilisateur ou argument invalides
 - fichier inexistant, erreur de lecture, format non-respecté
 - ...
 - Il faut éviter d'utiliser des exceptions pour gérer le flux normal d'exécution du programme en remplacement de boucles, **if-then**, etc.



Note : Déclencher une exception a un coût. Il provient majoritairement de l'instanciation de l'exception : durant cette instanciation, la trace de la pile d'exécution est établie. Source : **Java Performance Tuning (2nd edition)**, Jack Shirazi, O'Reilly, 2003



Bonnes pratiques

- **Ne pas capturer toutes les exceptions**
 - Il est recommandé de ne pas capturer « aveuglément » l'ensemble des exceptions qui peuvent survenir.
 - Il est préférable d'être sélectif dans la capture en spécifiant les classes que l'on veut capturer.

```
try {  
    /* code susceptible de déclencher une exception */  
} catch (Exception e) {  
}
```

- Au pire, si on ne sait pas / veut pas gérer une exception, mais qu'il faut satisfaire le compilateur, utiliser :

```
try {  
    /* ... */  
} catch (Exception e) {  
    throw new RuntimeException(e);  
    /* ou */  
    e.printStackTrace();  
}
```


Bonnes pratiques

- **Exceptions *non-contrôlées***

- Les **exception *non-contrôlées*** ne doivent généralement pas être capturées car elles signalent des **erreurs irrécupérables**.
- Exemple
 - La *division par zéro*, l'*accès en dehors des bornes* d'un tableau ou l'usage d'une *référence nulle* génèrent des *exceptions non-contrôlées*.

```
double x = 0;  
// --> ArithmeticException  
double y = 1/x;
```

```
int[] array = new int[10];  
// --> ArrayIndexOutOfBoundsException  
array[10] = 123;
```

Les *unchecked exceptions* signalent des *erreurs irrécupérables* (typ. des erreurs de programmation).
Il n'est généralement pas approprié de capturer ces exceptions !

- Pour cette raison, le compilateur ne force pas le programmeur à déclarer, capturer ou propager ces exceptions.
- Exception à cette règle : `NumberFormatException`

Table des Matières

- **Exception**

- Types
- Capture
- Déclenchement explicite
- Contrôlée ou pas ?
- Bonnes pratiques

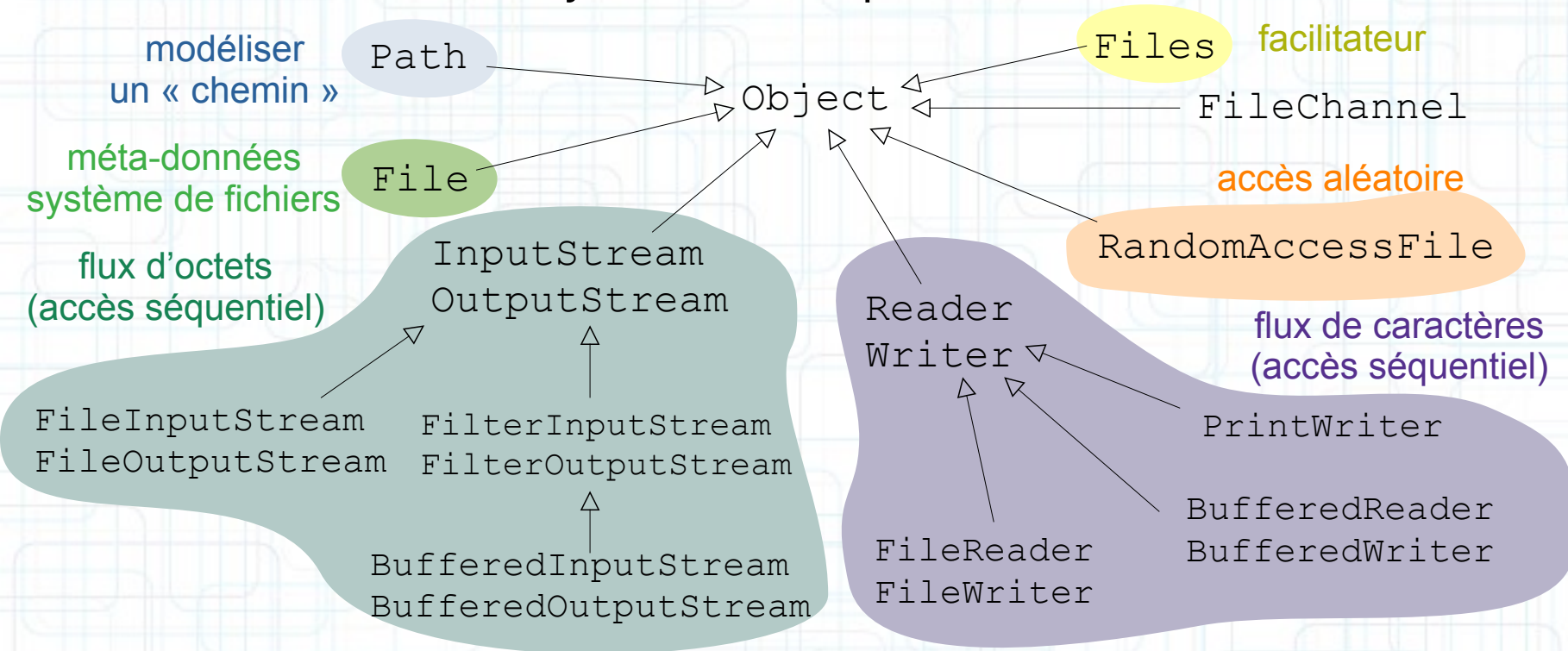
- **Fichiers**

- Accès séquentiel (flux)
- Accès aléatoire

Fichiers

- **I just want to read/write files !!...**

- La bibliothèque Java contient de nombreuses classes qui permettent d'accéder à ces données de différentes manières. Il est difficile de s'y retrouver de prime abord ...



- L'objectif de cette section est de comprendre quelles sont les classes appropriées à chaque cas d'utilisation.

Fichiers

- **I just want to read/write files !!...**
 - La classe `Files` ne définit que des méthodes de classe facilitant l'accès aux fichiers, telles `writeString()` et `readString()`.

```
Path path = Path.of("data/output.txt");  
Files.writeString(path, "Hello, World!",  
    StandardOpenOption.CREATE_NEW);
```

```
Path path = Path.of("/home/alfred/input.txt");  
String content = Files.readString(path);  
System.out.println("File content: " + content);
```

Option : crée le fichier s'il n'existe pas; échoue s'il existe déjà.

Par défaut:
`CREATE` écrase un fichier existant.

Fichiers

- **Interface Path**

```
/home/alfred/data/output.txt
```

- L'interface `Path` est destinée à représenter un « chemin », c'est-à-dire un moyen de localiser un fichier dans un système de fichiers. Un chemin est souvent composé de plusieurs parties
 - identifiant du **disque/système de fichier**
 - suite d'un ou plusieurs noms de **répertoires**
 - **nom du fichier** (éventuellement avec « extension »)
- Aperçu des méthodes
 - `of()` [de classe] crée un chemin à partir de `Strings`
 - `getFileName()` extrait le fichier
 - `isAbsolutePath()` teste si un chemin est absolu
 - `toAbsolutePath()` retourne un chemin absolu
 - etc.

Fichiers

- **Classe Files**

- Facilite les opérations communes de lecture écriture; destinées typiquement à de petits fichiers

- une chaîne de caractères

```
String readString(Path)  
Path writeString(Path, String, OpenOption...)
```

- liste de chaînes de caractères

```
List<String> readAllLines(Path)  
Path write(Path, Iterable<? extends CharSequence>,  
           OpenOption...)
```

- tableau d'octets

```
byte[] readAllBytes(Path)  
Path write(Path, byte[], OpenOption...)
```

- Des variantes des méthodes orientées « texte » permettent de spécifier l'encodage (par défaut UTF-8)

Fichiers

- **Classe Files**

- Facilite les opérations de manipulation du système de fichiers

- `createDirectory(Path, FileAttribute...)`
 - `exists(Path, LinkOption...)`
 - `delete(Path)`
 - `copy(Path, Path, CopyOption...)`
 - `getAttribute(Path, String, LinkOption...)`
et `setAttribute(Path, String, Object, LinkOption...)`
 - `size(Path)`
 - `etc.`

- Voir la documentation pour les détails...

Fichiers

- **Programmation fonctionnelle**

- La class `Files` permet également de retourner un `Stream` à partir d'un fichier texte.

```
Path p = Path.of("data/numbers.txt");  
var s = Files  
    .lines(p)  
    .limit(3)  
    .mapToDouble(Double::valueOf)  
    .sum(); /* 47.7715 */
```

```
42  
3.1415  
2.63  
17  
666
```

Table des Matières

- **Exception**

- Types
- Capture
- Déclenchement explicite
- Contrôlée ou pas ?
- Bonnes pratiques

- **Fichiers**

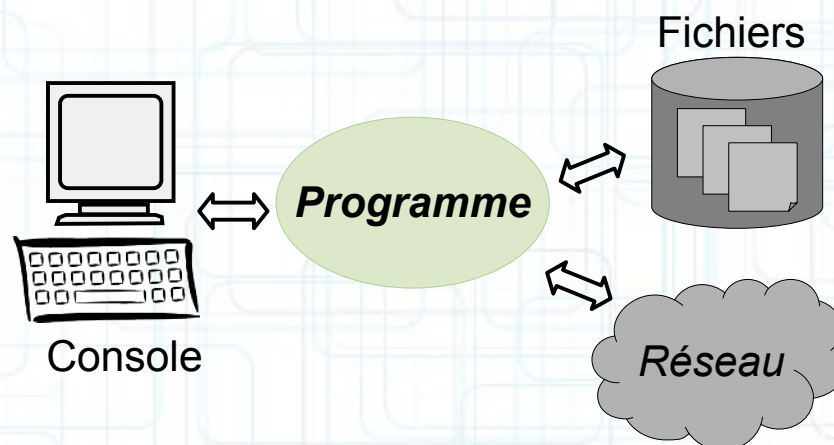
- Accès séquentiel (flux)
- Accès aléatoire

Fichiers

- **Flux d'entrées/sorties**

- Les **sources** et **destinations** de données d'un programme peuvent être de types très divers

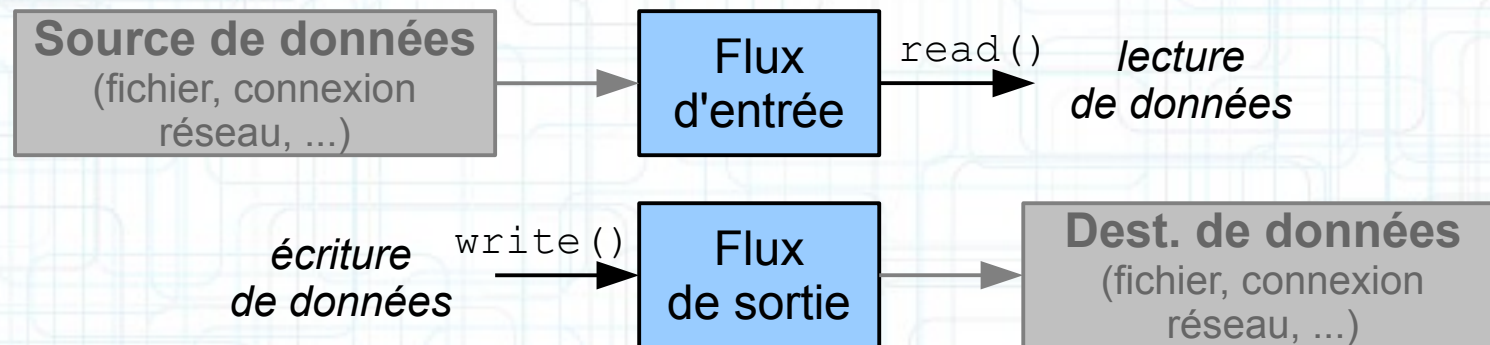
- fichiers
- console
- autres programmes
- connexions réseaux
- ...



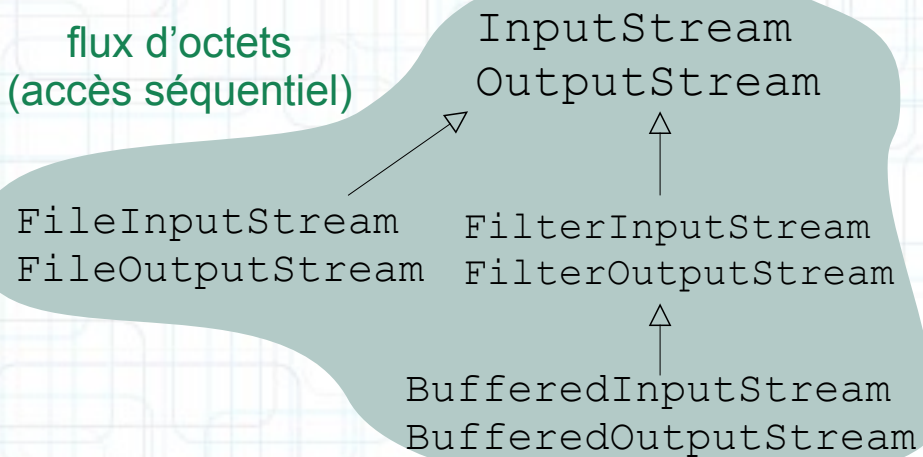
- L'**accès** à ces sources et destinations de données peut être réalisé de différentes façons
 - accès séquentiel, accès aléatoire, accès avec tampon mémoire (*buffer*), ...
 - accès binaire, caractère, par ligne, par mot, par enregistrement, ...

Fichiers

- Accès séquentiels (flux)

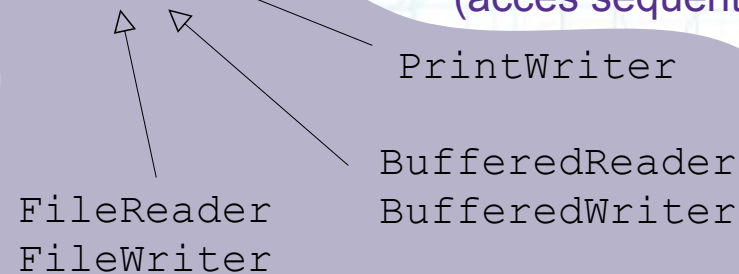


flux d'octets
(accès séquentiel)



Reader
Writer

flux de caractères
(accès séquentiel)



Fichiers

- Pour les étudiants pressés...

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class SimplestFileAccess {

    public static void main(String [] args) {
        try (BufferedReader r =
            new BufferedReader(new FileReader("toto.txt")))
        {
            String s;
            while ((s= r.readLine()) != null)
                System.out.println(s);
        } catch (IOException e) {
            System.err.println(e.getMessage());
        }
    }
}
```

Afin d'améliorer les performance, permet la lecture au travers d'un tampon mémoire.

Ouvre un fichier et le considère comme un flux de caractères en lecture.

Ce fichier sera fermé automatiquement par la directive **try-with-resource**.

Lit une ligne à partir du fichier et la retourne sous forme d'une chaîne de caractères.

Si la fin du fichier est atteinte, retourne **null**.

Fichiers

- **Clause *try-with-resources***

- Depuis la version 7 de java, une nouvelle forme de directive **try** est introduite, adaptée à la gestion d'erreurs en présence de ressources qui nécessitent une libération après usage.
- La nouvelle directive, nommée **try-with-resources**, garantit la fermeture de ces ressources.
- Pour qu'une ressource puisse être utilisée dans une directive **try-with-resources**, il faut qu'elle implémente la nouvelle interface `java.lang.AutoCloseable` (définissant une méthode unique `close()`)

```
public static void main(String [] args) throws Exception
{
    try (InputStream is= new FileInputStream("/tmp/data.bin")) {
        long count= 0;
        while (is.read() >= 0)
            count++;
    }
}
```

Fichiers

- **Clause `try-with-resources`**

- Le même type de construction est possible en Python (depuis la version 2.5) sous la forme `with...as` comme illustré dans l'exemple ci-dessous

```
with open("/var/log/messages", "r") as myFile :  
    for line in myFile :  
        print line
```

- Les objets fichier en Python sont équipés de méthodes `__enter__` et `__exit__` qui sont appelées respectivement lors de l'entrée et de la sortie dans le bloc `with...as`. La méthode `__exit__` peut ainsi s'occuper de fermer le fichier.

Fichiers

- **Flux d'octets**

- Les classes abstraites `InputStream` et `OutputStream` fournissent chacune une seule méthode importante : respectivement `read()` et `write()`

```
public abstract class InputStream {  
    public int available() { ... }  
    public void close() { ... }  
    public abstract int read();  
    public int read(byte [] b) { ... }  
    public int read(byte [] b, int off, int len) { ... }  
    public long skip(long n) { ... }  
}
```

```
public abstract class OutputStream {  
    public void close() { ... }  
    public int flush() { ... }  
    public abstract void write(int b);  
    public void write(byte [] b) { ... }  
    public void write(byte [] b, int off, int len) { ... }  
}
```


Fichiers

- **Flux de caractères**

- Les classes abstraites `Reader` et `Writer` définissent chacune une seule méthode importante : respectivement `read` et `write`.

```
public abstract class Reader {  
    public abstract void close();  
    public int read();  
    public abstract int read(char [] cbuf, int off, int len);  
    public boolean ready() { ... }  
    public long skip(long n) { ... }  
}
```

```
public abstract class Writer {  
    public Writer append(char c) { ... }  
    public abstract void close();  
    public abstract void flush();  
    public void write(int c) { ... }  
    public abstract void write(char [] cbuf, int off, int len);  
    public void write(String str) { ... }  
}
```

Table des Matières

- **Exception**
 - Types
 - Capture
 - Déclenchement explicite
 - Contrôlée ou pas ?
 - Bonnes pratiques
- **Fichiers**
 - Accès séquentiel (flux)
 - Accès aléatoire

Fichiers

- **Accès aléatoire**
 - Non couvert cette année.